Lesson 12 — Robot Equations of Motion

Module 3: Dynamics & Control | Accelerated lesson | Status: Completed

Learning outcomes

- Explain how motor torque, inertia, damping and gravity determine angular acceleration.

- Simulate a one-joint arm using semi-implicit Euler integration.

- Distinguish zero motor torque from torque that compensates gravity at a particular configuration.

1. From kinematics to dynamics

Kinematics describes where a joint and end-effector move. Dynamics models *why* they move, including applied torque, gravity, inertia, and damping.

For a simplified one-joint arm moving in a vertical plane, with angle theta measured counterclockwise from the horizontal:

```
I * theta_ddot + b * theta_dot + m*g*L*cos(theta) = tau
```

Rearranging gives the angular acceleration:

```
theta_ddot = (tau - b*theta_dot - m*g*L*cos(theta)) / I
```

Definitions:

| Symbol | Meaning | Unit |

| --- | --- | --- |

| I | Constant moment of inertia about the joint | kg m^2 |

| theta | Joint angle measured from horizontal | radians |

| theta_dot | Angular velocity | rad/s |

| theta_ddot | Angular acceleration | rad/s^2 |

| b | Viscous damping coefficient | N m s/rad |

| m | Point-mass payload | kg |

| g | Gravitational acceleration | m/s^2 |

| L | Distance from pivot to payload | m |

| tau | Motor torque | N m |

Model assumptions: single rigid, fixed-axis joint; constant inertia; a point-mass payload at distance L; simplified linear viscous damping; no actuator limits, collisions, gear dynamics, or link self-weight unless included in I. This is a teaching model, not a full robot dynamics model.

2. Physical interpretation

- Motor torque tau: applied torque accelerates the joint in its positive direction.

- Gravity m*g*L*cos(theta): gravity pulls the arm downward; its torque depends on orientation.

- Damping b*theta_dot: resists angular velocity.

- Inertia I: greater inertia produces less angular acceleration for the same net torque.

At the horizontal position (theta=0), the gravitational torque magnitude for m=1 kg and L=0.5 m is 4.905 N m. The arm will initially accelerate downward if no motor torque is applied. If the motor applies +4.905 N m and the arm starts at rest, the net torque is zero and this exact horizontal state remains stationary in the ideal model.

Important limitation: a constant 4.905 N m torque is not gravity compensation for every angle. Angle-dependent gravity compensation would use tau_g(theta)=m*g*L*cos(theta).

3. Numerical integration

At each timestep, compute angular acceleration from the current joint angle and velocity. With semi-implicit Euler:

```
omega_next = omega + alpha*dt

theta_next = theta + omega_next*dt
```

This creates a time trajectory. Numerical integration approximates a continuous physical system; its accuracy depends on timestep and method.

4. Full Python practical

Save as `lesson_12_equations_of_motion.py`:

```
import numpy as np
import matplotlib.pyplot as plt

I = 0.5       # kg*m^2
b = 0.2       # N*m*s/rad
m = 1.0       # kg
L = 0.5       # m
g = 9.81      # m/s^2

dt = 0.005
duration = 1.0
steps = int(duration / dt)

def simulate(motor_torque):
    theta = 0.0
    omega = 0.0
    angles = [theta]
    for _ in range(steps):
        gravity_torque = m * g * L * np.cos(theta)
        alpha = (motor_torque - b * omega - gravity_torque) / I
        omega += alpha * dt
        theta += omega * dt
        angles.append(theta)
    return np.rad2deg(np.array(angles))

without_motor = simulate(0.0)
holding_torque = m * g * L
with_motor = simulate(holding_torque)
time = np.linspace(0, duration, steps + 1)

print(f"Holding torque: {holding_torque:.3f} N*m")
print(f"No motor final angle: {without_motor[-1]:.2f} deg")
print(f"Holding motor final angle: {with_motor[-1]:.2f} deg")
assert without_motor[-1] < -1.0
assert np.allclose(with_motor, 0.0, atol=1e-8)
print("Verification: PASSED")

plt.plot(time, without_motor, label="No motor torque")
plt.plot(time, with_motor, label="Gravity compensation at theta=0")
plt.xlabel("Time (s)")
plt.ylabel("Joint angle (degrees)")
plt.title("Lesson 12: Single-Joint Dynamics")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("lesson_12_joint_dynamics.png", dpi=160)
plt.show()
```

Run with `python lesson_12_equations_of_motion.py` in an environment with numpy and matplotlib.

5. Expected observations and verification

| Test | Expected behavior |

| --- | --- |

| `tau = 0` | The arm initially rotates downward from its horizontal starting state. |

| `tau = m*g*L` | The arm remains horizontal when initially at rest. |

| Verification | `PASSED` |

| Output artifact | `lesson_12_joint_dynamics.png` |

Learner-reported outcome: Python experiment passed. This is based on the learner's confirmation; the actual terminal log and plot were not uploaded.

6. Assessment record

Question B: What does the equation of motion calculate from the current state and motor torque? A — Angular acceleration. Correct.

Question C: Why does the arm stay horizontal under gravity compensation in this simulation? B — The torque balances gravity and the arm starts at rest. Correct.

Lesson 12 status: PASSED, including Python experiment and both conceptual questions.

7. Practical engineering notes

- Torque is in N m, while inertia is in kg m^2; angular acceleration is rad/s^2.

- Gravity compensation is model- and configuration-dependent.

- Real robot joints also experience friction, actuator saturation, linkage inertia, uncertainty and disturbances.

- The next lesson introduces feedback control to automatically regulate joint position rather than using only a fixed torque.

8. Self-review

1. If the motor applies no torque to a horizontal arm at rest, which direction does gravity initially accelerate it?

2. Why does more rotational inertia reduce acceleration for the same net torque?

3. Why does fixed holding torque only exactly balance gravity at certain angles?

4. What is the difference between an equation of motion and the numerical integration method used to solve it?

Next lesson: Lesson 13 — Feedback Control and PID Basics.